

Taller 2: “Creación de un Videojuego 3D”

Infested City Run

1.- Objetivo:

Diseñar e implementar un prototipo de videojuego 3D, utilizando Unity, C# y aplicando conocimientos de programación orientada al objeto.

2.- Enunciado:

Se solicita crear un videojuego 3D para computador del estilo Infinite Runner Shooter llamado “Infested City Run”, el cual consiste en un jugador que está sumergido en medio del apocalipsis en una gran ciudad infinita donde debe correr hacia adelante sin detenerse, disparando a los infectados y esquivando obstáculos que van apareciendo en el camino de tres carriles.



Figura 1: Idea general. Jugador disparando balas en un camino con obstáculos y enemigos.

★ Jugador

El Jugador debe tener las siguientes funcionalidades:

- Moverse en 2 direcciones horizontales (izquierda y derecha) con las teclas “A” y “D”
- Saltar hacia arriba (distancia suficiente que permita pasar por arriba de autos)
- Movimiento hacia adelante forzado sin detenerse (correr por defecto)
- Disparar proyectiles (balas) hacia adelante al presionar la tecla “F”
- Recolectar monedas
- Reiniciar la partida al colisionar con enemigos u obstáculos



Figura 2: Ejemplo de Jugador y su recorrido en el camino

★ Mapa y Partida

El mapa debe estar conformado con una serie de plataformas que se repetirán infinitamente en las cuales se encontrarán enemigos, obstáculos y monedas. Puede decorarse agregando elementos a los lados de cada plataforma (dentro de su jerarquía de objeto) como edificios, muros, árboles, etc.

No existe forma de ganar el juego, el desafío es llegar cada vez más lejos en cada intento.

Fondo: La partida debe tener un color o imagen de fondo.

Límites del mapa: Deben existir límites a la izquierda y derecha del camino de tal manera que no se pueda salir del mapa por los lados de ninguna forma.

Resolución: 1980 x 1080

Cámara: Sigue al Jugador posicionándose por arriba y detrás del jugador mientras avanza con él. No se mueve de forma horizontal, sólo se mueve hacia adelante junto con el jugador.

Plataforma (Suelo Plano): Debe haber una serie de plataformas (planas) a modo de suelo firme donde el jugador pueda correr. Estas plataformas deben ir formando un camino largo y simular que es infinito con un sistema de generación de suelos. Cada plataforma debe tener Material y Color.

Cada Plataforma debe tener como mínimo 1 elemento “hijo” (en su jerarquía de objetos) que aparezca en ella al generarse: una moneda, un enemigo o un obstáculo. Cuando una Plataforma se elimine su “hijo” también lo hará con ella.

Generación de plataformas (“suelos infinitos”): Cada plataforma detrás del jugador se irá eliminando a la vez que se irán creando nuevas plataformas en su lugar al final del mapa, de tal forma que se pueda hacer una carrera infinita (ver apartado “Generación de Plataformas”)

★ Generación del Plataformas:

Al comenzar la partida se deben generar una serie de plataformas que actúen como suelo (10 mínimo, 30 máximo) y se destruirán aquellas por las que el jugador ya pasó (cuando dejan de estar colisionando con él, por ejemplo).

Cada vez que se destruye una plataforma, se genera otra al final del mapa (la última en la serie de suelos) tal como se muestra en la Figura 3. La idea es mantener una serie de plataformas formando un camino bastante extenso por delante del jugador mientras éste último avanza. Logrando así un camino infinito que: destruye 1 suelo cuando el jugador lo deja atrás, gatillando generar 1 suelo.

Se recomienda realizar un Prefab de Suelo al que se le instancie otro Prefab igual en su borde.

En cada Plataforma, hay 3 posiciones posibles donde un obstáculo, enemigo y moneda puede aparecer.

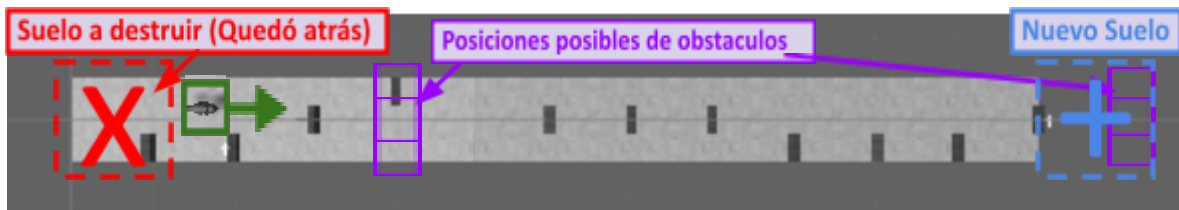


Figura 3: Creación y destrucción de las plataformas con obstáculos mientras el jugador avanza.

★ Enemigos y Obstáculos

Todos los enemigos y obstáculos deben tener este comportamiento en común:

- Matar al jugador al colisionar con él (reiniciando la partida)
- Destruirse al recibir un proyectil (bala) del jugador.
- Aparecer de forma aleatoria en uno de los tres carriles del camino

Cada enemigo y obstáculo puede aparecer en 3 posibles posiciones de una plataforma cuando esta última es generada. Aparece en una posición aleatoria entre estas 3.

Los obstáculos siempre estarán quitos y fijos a la plataforma en donde aparecieron, en cambio los enemigos siempre estarán en movimiento en una sola dirección después de haber aparecido en su respectiva plataforma.

Cada obstáculo y enemigo debe tener su respectivo Material asociado a su Modelo.

Zombie: Enemigos que se mueven en línea recta hacia adelante sin parar en dirección contraria al jugador ("hacia la pantalla"). Este tipo de enemigo aparece en uno de los tres carriles de forma aleatoria. Sólo se mantiene en su carril (sin cambiarse a otro) avanzando en línea recta infinitamente y el Jugador no debe poder saltarlo por encima.



Figura 4: Ejemplo de Enemigo Zombie y su recorrido en el camino

Vehículo: Obstáculo estático que aparece en uno de los tres carriles de forma aleatoria, este debe poderse saltar sin problemas. Deben haber como mínimo 3 tipos de autos diferentes, donde sólo debe cambiar su color o modelo (se acepta cualquier tipo de vehículo).



Figura 5: Ejemplo de Obstáculo Vehículo y su posición estática en el camino

★ Coleccionables y Puntaje (UI)

Todo puntaje debe reiniciarse cuando el jugador pierde la partida.

Puntajes:

- Debe haber un contador de monedas anclado a la pantalla en la parte superior de esta (siempre visible) que aumenta cuando el jugador recolecta una moneda.
- Debe haber un contador de puntaje anclado a la pantalla en la parte superior de esta (siempre visible), el cual incrementa según la distancia que va avanzando el jugador.

Moneda: Debe existir una Moneda recolectable por el jugador al colisionar con ella que aumente el contador de monedas.

★ **Proyectiles**

Los proyectiles lanzados por el Jugador deben poder destruir a un Enemigo.

Todos los proyectiles deben destruirse al colisionar con los Obstáculos y Enemigos. No son afectados por la gravedad.

Estos proyectiles deben poder lanzarse en un intervalo de mínimo 0.5 segundos (para no lanzar proyectiles en grandes cantidades).

★ **Menú de derrota**

Al perder la partida debe aparecer un Menú de derrota con un botón para reiniciar la partida. El jugador pierde cuando colisiona con un enemigo u obstáculo.

★ **Botones**

En cada tipo de Menú, cada botón debe ser funcional y tener interacciones básicas como una animación (ya sea que cambie de color, tamaño o aparezca una imagen) al presionar el botón y al pasar el ratón por encima del botón.

★ **Audio**

La música y los sonidos son parte fundamental de un videojuego, por lo cual, debe haber una música y sonido en acorde a cada situación:

- Sonido de Jugador al disparar.
- Sonido de Enemigo al morir.
- Música de fondo del juego.
- Música al perder la partida (Menú de derrota).

★ **Animaciones**

Deben existir las siguientes animaciones, las cuales vienen incluidas dentro de los archivos subidos a plataforma (vienen dentro de cada Modelo):

- **Jugador** (animación de movimiento en ejecución infinita)
- **Enemigo** (animación de movimiento en ejecución infinita)

★ **Jugabilidad**

El juego se debe considerar "jugable" al menos 30 seg. considerando cuidadosamente lo siguiente:

- Velocidades de Jugador, Proyectiles y Enemigos (en lo posible favorecer al Jugador)
- Cantidad de enemigos y obstáculos por plataforma.
- Tamaño de Jugador, Enemigos y Plataforma.

Para una buena jugabilidad, se recomienda que el videojuego sea probado por más de una persona

★ Código

Todo Script y/o Clase creada debe estar documentada debidamente explicando qué hace cada atributo, método, clase y para qué se utilizan.

Cada variable debe tener un nombre comprensible según su función (que tengan sentido).

Sólo usar el lenguaje de programación C# para la implementación de la solución.

Se pueden crear más variables, métodos, clases y Scripts para el desarrollo de algún problema con tal de cumplir lo solicitado.

Todo el código escrito será explicado a través del informe. Debe ser coherente con el Modelo de Dominio y Diagrama de Clases que ha realizado en el informe.

★ Archivos, Materiales y Modelos 3D

Los Archivos y Assets necesarios para el desarrollo del proyecto serán subidos en un archivo Rar a la plataforma, se permite buscar modelos o diseños extra siempre y cuando cumpla con lo requerido en el enunciado. Los audios deben buscarlos por su cuenta (libertad creativa).

Los modelos necesarios para jugador, enemigo y obstáculos serán entregados en el archivo Rar, en el caso de los Materiales, estos estarán incluidos en cada Modelo. En caso de no tener material, este debe ser creado pudiendo utilizar imágenes o colores para lograr recrear algo similar a las imágenes adjuntas en este documento.

Es necesario que cada Modelo que se adjuntó sea desempaquetado luego de incluirlo dentro del Proyecto, suelen contener ya incluidos sus respectivas animaciones, materiales, texturas y mallas.

3- Resultado:

Como resultado, debe quedar una aplicación parecida a la imagen adjunta más abajo:



Figura 6: Partida de Infested City Run con sus elementos señalados.

4.- Entregas: 7 de Junio.

Cada entrega debe **incluir Nombre y RUT** de cada integrante o tendrá descuento.

I.- Informe del Taller: (30%)

Se debe subir a la plataforma un **Informe Word** que tenga Portada (asignatura, nombre y rut de los integrantes), Introducción, Desarrollo y Conclusión, explicando el **Modelo del Dominio y Diagrama de Clases** que utilizarán para desarrollar el proyecto y cómo llevarán a cabo la implementación de este. Deben considerar todos los elementos que suele tener un modelo del dominio y un diagrama de clases y explicar la manera en que realizaron el Taller (cómo funcionan las mecánicas más importantes, métodos utilizados y variables importantes, GameObjects importantes con su comportamiento y componentes, contenido extra si es que lo incluyeron, carpetas/estructura del proyecto, assets utilizados, etc). Debe tener un enlace al repositorio (público) en Github.

II.- Proyecto en Unity (70%)

Dentro del informe subido a plataforma, debe estar el **enlace al repositorio Github** donde debe estar el proyecto del taller completado. El **repositorio debe estar público** al entregarlo.

Debe seguir el siguiente formato e indicaciones para las entregas (incluyendo RUT y Nombre):

- **Archivo Informe:** Taller2_RutIntegrante1_RutIntegrante2.docx (*reemplazar rut*)
- **Repositorio:** En un archivo README.md o en la descripción del repositorio debe incluir **Nombre y RUT** de cada integrante (puede crear el README.md desde Github).

5.- Observaciones:

- Ante cualquier duda, contactar al ayudante samuel.fuentes@alumnos.ucn.cl
- El taller se realiza en **grupo máximo de 2 personas**.
- La versión de Unity **debe** ser la **2022.3.62f3**.
- Obtendrán **1 décima adicional a su nota final por cada complemento extra** que realicen (menús, sonidos, animaciones, mecánicas, enemigos, objetos, etc.) (máximo 10 décimas).
- El motor gráfico debe ser Unity, el código en C# y proyecto debe ser bajo una plantilla 3D.
- Pueden agregar más entidades y atributos a su modelo de dominio y diagrama de clases que no estén presentes en el enunciado siempre que estén implementados y tengan completo sentido con el juego. Debe haber coherencia entre Taller, Informe y Diagramas.
- Deben utilizar los assets proporcionados en el archivo Rar (pueden añadir más)

6.- Penalizaciones:

- Se descontarán **10 décimas** a la nota por cada día de retraso en la entrega
- Entregas sin indicar nombre y rut de cada integrante tendrán **5 décimas** de descuento
- Palabras, sonidos o imágenes denigrantes, inapropiadas u ofensivas **nota máxima 4.0**
- Si su proyecto no funciona/compila, la **nota máxima 5.0**
- La copia de talleres será sancionada **con nota 1.0**
- Si su repositorio no está público en la revisión, se descontará **5 décimas**